

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 17 · Flujos de texto: Reader, Writer y try-with-resources

Lectura de ficheros de texto: FileReader, BufferedReader, Scanner

Escritura: FileWriter, BufferedWriter, PrintWriter

try-with-resources y AutoCloseable

Codificación de caracteres y NIO (Files.newBufferedReader)

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Entrada/salida de texto

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué clase envuelve un `FileReader` para leer el fichero línea a línea con eficiencia?
- a) `BufferedReader`
 - b) `DataInputStream`
 - c) `ObjectReader`
 - d) `LineReader`

RESPUESTA / SOLUCIÓN

a) `BufferedReader` — Añade un buffer de memoria y el método `readLine()`, evitando una lectura del disco por carácter.

-
2. ¿Qué método de `BufferedReader` lee una línea completa?
- a) `readLine()`
 - b) `read()`
 - c) `nextLine()`
 - d) `line()`

RESPUESTA / SOLUCIÓN

a) `readLine()` — Devuelve la línea sin el salto de línea, o null cuando se llega al final del fichero.

-
3. ¿Qué clase combina buffer y métodos `print()/println()` para escribir texto cómodamente?

- a) PrintWriter
- b) FileWriter
- c) DataWriter
- d) BufferedOutput

RESPUESTA / SOLUCIÓN

a) PrintWriter — Ofrece print()/println() sobre cualquier Writer y no lanza IOException en esos métodos.

4. ¿Qué construcción cierra automáticamente los recursos al final del bloque?

- a) try-with-resources
- b) try-catch normal
- c) finally manual
- d) try-alone

RESPUESTA / SOLUCIÓN

a) try-with-resources — Declara los recursos entre paréntesis y los cierra automáticamente al salir del bloque.

5. ¿Qué interfaz deben implementar los recursos usados en try-with-resources?

- a) AutoCloseable
- b) Serializable
- c) Resource
- d) Disposable

RESPUESTA / SOLUCIÓN

a) `AutoCloseable` — (o su subinterfaz `Closeable`). Sin ella el compilador rechaza el recurso.

6. **`FileWriter`, por defecto, sobrescribe el contenido del fichero si ya existe.**

RESPUESTA / SOLUCIÓN

Verdadero — Por defecto abre en modo escritura (trunca el fichero); hay que pasar el flag `true` al constructor para añadir al final (`append`).

7. **¿Cuál es la forma correcta de leer un fichero con `Scanner`?**

- a) `new Scanner(new File("datos.txt"))`
- b) `Scanner.readFile("datos.txt")`
- c) `File.scanner("datos.txt")`
- d) `Scanner.open("datos.txt")`

RESPUESTA / SOLUCIÓN

a) `new Scanner(new File("datos.txt"))` — `Scanner` acepta un `File` (o `Path`) y permite leer con `nextLine()`, `nextInt()`, etc.

8. ¿Qué clase se recomienda para leer un fichero de texto con codificación explícita (p. ej. UTF-8)?
- a) `Files.newBufferedReader(path, StandardCharsets.UTF_8)`
 - b) `new BufferedReader(new FileReader(path))` sin charset
 - c) `new DataReader(path)`
 - d) `new TextFile(path)`

RESPUESTA / SOLUCIÓN

a) `Files.newBufferedReader(path, charset)` — Permite indicar la codificación; `FileReader` sin charset usa la codificación por defecto de la plataforma.

-
9. ¿Qué método de `PrintWriter` escribe sin salto de línea al final?
- a) `print()`
 - b) `println()`
 - c) `writeLine()`
 - d) `newLineOnly()`

RESPUESTA / SOLUCIÓN

a) `print()` — Escribe el texto sin añadir salto de línea; `println()` lo añade.

-
10. Si no se cierra un `BufferedWriter`, parte del contenido puede quedarse en el buffer y no escribirse en el fichero.

RESPUESTA / SOLUCIÓN

Verdadero — Al cerrar (o hacer `flush()`) se vacía el buffer al disco; si el programa termina sin cerrar, puede perderse contenido.

Parte 2: Análisis y depuración

3 puntos

1. El siguiente código lee un fichero carácter a carácter. ¿Qué problemas tiene? Propón una versión mejorada. (1,5 pts)

```
import java.io.*;\npublic class LeerLento {\n    public static void main(String[] args) throws IO
```

RESPUESTA / SOLUCIÓN

1. No se cierra el recurso: si ocurre una excepción, el FileReader queda abierto (fuga de recursos).
2. read() sin buffer hace una llamada al sistema por carácter → muy lento en ficheros grandes.

Solución: try-with-resources + BufferedReader y readLine():

```
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {\n    String linea;\n    while ((linea = br.readLine()) != null) System.out.println(linea);\n}
```

2. Este programa copia un fichero línea a línea pero pierde líneas. ¿Por qué? Corrígelo. (1,5 pts)

```
import java.io.*;\npublic class CopiaBug {\n    public static void main(String[] args) throws IOE
```

RESPUESTA / SOLUCIÓN

El bucle llama a br.readLine() dos veces por iteración: una en la condición y otra dentro del cuerpo.

La primera línea se lee y se descarta (solo se comprueba que no es null), y la segunda se escribe.

Resultado: se pierden las líneas impares y la última línea puede escribirse como null.

Corrección: guardar la línea en una variable:

```
String linea;\nwhile ((linea = br.readLine()) != null) { pw.println(linea); }
```

Parte 3: Ejercicios de programación

6 puntos

1. Escribe un programa que lea un fichero notas.txt (formato: nombre;nota por línea) y muestre la nota media de la clase. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.io.*;

public class MediaNotas {

    public static void main(String[] args) {

        double suma = 0;

        int n = 0;

        try (BufferedReader br = new BufferedReader(new FileReader("notas.txt"))) {

            String linea;

            while ((linea = br.readLine()) != null) {

                String[] partes = linea.split(";");

                suma += Double.parseDouble(partes[1].trim());

                n++;

            }

        } catch (IOException e) {

            e.printStackTrace();

        }

        System.out.println("Media: " + (n > 0 ? suma / n : 0));

    }

}
```

2. Escribe un programa que genere un fichero tabla.txt con la tabla de multiplicar de un número leído por teclado, usando PrintWriter. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.io.*;
import java.util.Scanner;

public class TablaMultiplicar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Número: ");
        int n = sc.nextInt();
        try (PrintWriter pw = new PrintWriter(new FileWriter("tabla.txt"))) {
            for (int i = 1; i <= 10; i++) {
                pw.println(n + " x " + i + " = " + (n * i));
            }
            System.out.println("Tabla escrita en tabla.txt");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

3. Copia un fichero de texto línea a línea usando try-with-resources y Files.newBufferedReader con UTF-8. Si el fichero de origen no existe, muestra un mensaje sin romper el programa. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.io.*;

public class CopiaTexto {
    public static void main(String[] args) {
        Path origen = Paths.get("origen.txt");
        Path destino = Paths.get("copia.txt");
        try (BufferedReader br = Files.newBufferedReader(origen, StandardCharsets.UTF_8);
            PrintWriter pw = new PrintWriter(Files.newBufferedWriter(destino,
                StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = br.readLine()) != null) pw.println(linea);
            System.out.println("Copiado");
        } catch (NoSuchFileException e) {
            System.out.println("El fichero de origen no existe");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Parte 4: Pregunta teórica

2 puntos

1. Explica la diferencia entre flujos de bytes (InputStream/OutputStream) y flujos de caracteres (Reader/Writer). ¿Por qué conviene usar BufferedReader y PrintWriter?

RESPUESTA / SOLUCIÓN

Los flujos de bytes trabajan con datos binarios (byte a byte): sirven para imágenes, audio, objetos serializados, etc.

Los flujos de caracteres (Reader/Writer) traducen bytes a caracteres usando una codificación (UTF-8, ISO-8859-1...): adecuados para texto.

BufferedReader añade un buffer y readLine(), reduciendo las llamadas al disco de una por carácter a una por bloque.

PrintWriter ofrece print()/println() sobre Writer y no obliga a capturar IOException en cada escritura, simplificando el código de salida de texto.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)